



Security Assessment

Final Report



Gold Strategy

March 2026

Prepared for TradePort

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	4
Threat and Security Overview.....	5
System overview.....	5
Threat surface.....	5
Actors.....	5
Assets.....	5
Actions.....	6
Core threat model.....	6
Findings Summary.....	7
Severity Matrix.....	7
Detailed Findings.....	8
Medium Severity Issues.....	10
M-01: An attacker can steal un-swapped strategy fees using swap router's refund mechanism.....	10
M-02: Protocol fees are wrongly determined resulting in both liquidity providers and protocol being credited the wrong portion of the swap fees.....	12
M-03: Strategy balance swap can be sandwiched due to the function being permissionless.....	13
Low Severity Issues.....	15
L-01: An attacker can grief reward accrual by poking at a position regularly resulting in no rewards emitted.....	15
L-02: Strategy fee internal swap will be affected by the initial swap's price impact.....	17
L-03: Router exact output functions fail to ensure the output coin value matches the expected value...	19
Informational Severity Issues.....	21
I-01: Un-swapped strategy payment will be put in reserves when paying strategy fees.....	21
I-02: apply_strategy can halt after changing strategy Coin.....	22
I-03: Incoherent dynamic field addition when revoking pool manager.....	23
I-04: Missing natspec for the pair parameter in position manager module.....	24
I-05: Unnecessary if/else statement during fee withdrawal.....	25
Disclaimer.....	26
About Certora.....	26

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Gold Strategy	https://github.com/Certora/tradeport-gold-strategy/	cd12bf7	Sui

Project Overview

This document describes the security review of **Gold Strategy**. The work was undertaken from **February 16th, 2026** to **March 16th, 2026**.

The following contract list is included in our scope:

tradeport-gold-strategy/tradeport_gold_strategy/sources/*

tradeport-gold-strategy/tradeport_strategy_clmm/sources/*

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Tradeport Gold Strategy is a Sui-based protocol centered on **GLDSTR**, a coin backed by **XAUM**, and supported by an integrated concentrated liquidity market maker. The system combines a strategy module responsible for minting, redemption, and treasury management with a CLMM layer that handles swaps, liquidity positions, fee accounting, and strategy-fee accrual.

Value generated through market activity is redirected into the strategy's backing asset. Fees collected through the liquidity layer may be converted and incorporated into the strategy treasury, increasing the reserves that support **GLDSTR**.

As a result, **GLDSTR** is designed to generate yield through additional **XAUM** accumulation, giving holders increasing exposure to the underlying gold-linked asset over time.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

System overview

The system is composed of two tightly coupled layers.

The first is the strategy layer in `tradeport_gold_strategy`, which manages `GLDSTR` minting, redemption, treasury balances, fee collection, and conversion of accrued value into `XAUM`, the gold-linked backing asset. The second is the CLMM layer in `tradeport_strategy_clmm`, which manages pool creation, swaps, liquidity positions, fee accounting, reward emissions, and strategy-fee accrual.

In operational terms, users interact with pools and positions at the CLMM layer, while `GLDSTR` holders depend on the strategy layer to preserve and grow the `XAUM` backing.

Threat surface

Actors

Who can do what.

- **CLMM admin** (`AdminCap` owner): can change protocol and strategy parameters at the CLMM level and collect protocol fees
- **Gold Strategy admin** (`manager.admin` address): can create the strategy, set fees, update pool references, change admin state, lock functionality, and withdraw manager fees
- **Permissionless external users and contracts**: can call CLMM entrypoints such as swap, liquidity management, fee collection, reward claims, `apply_strategy`, and `GLDSTR` redemption

Assets

What has value or security importance.

- **Pool economic balances**: `reserve_x`, `reserve_y`, `strategy_reserve_x`, `strategy_reserve_y`, `protocol_fee_x`, `protocol_fee_y`
- **Position claim assets**: `coins_owed_x`, `coins_owed_y`, and reward debt
- **Gold strategy backing assets**: `manager.xaum` and `manager.fee`

- **Control rights and control-plane state:** AdminCap, manager.admin, Versioned, pause state, version state, and strategy UID authorization
- **Settlement assets:** SwapReceipt and FlashReceipt

Actions

Impactful state transitions and value computations.

- **Swap and settlement path:** compute amounts and fees in swap_internal, then settle through pay
- **Fee split logic:** distribute value across strategy, protocol, and LP accounting paths
- **Strategy fee conversion path:** convert accrued strategy fees, including nested internal swap behavior
- **Flash loan path:** issue assets through flash and enforce repayment through repay
- **Liquidity position lifecycle:** open, increase, decrease, and collect positions and fees
- **Pool creation and initialization:** determine canonical pool instances and initial price state
- **Gold strategy actions:** create GLDSTR, apply the strategy, and redeem for XAUM
- **Rounding-sensitive withdrawal path:** clipping behavior through safe_withdraw.

Core threat model

The review considered the following attack classes:

- access-control failures on privileged configuration and pool-creation paths
- accounting mismatches between swap execution, receipts, and settlement
- partial-fill behavior in both user-facing and internal conversion flows
- value loss through refunds, donations, or reserve misattribution
- incorrect fee allocation between LPs, protocol, and strategy
- reward-accounting drift, truncation, or underfunding
- boundary-price and low-liquidity conditions
- economic manipulation of permissionless strategy-triggering flows
- cross-module risk where strategy
- owned balances rely on generic router behavior

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	3	3	3
Low	3	2	2
Informational	5	5	5
Total	11	10	10

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
M-01	An attacker can steal un-swapped strategy fees using swap router's refund mechanism	Medium	Fixed
M-02	Protocol fees are wrongly determined resulting in both liquidity providers and protocol being credited the wrong portion of the swap fees	Medium	Fixed
M-03	Strategy balance swap can be sandwiched due to the function being permissionless	Medium	Fixed
L-01	An attacker can grief reward accrual by poking at a position regularly resulting in no rewards emitted	Low	Fixed
L-02	Strategy fee internal swap will be affected by the initial swap's price impact	Low	Acknowledged
L-03	Router exact output functions fail to ensure the output coin value matches the expected value	Low	Fixed
I-01	Un-swapped strategy payment will be put in reserves when paying strategy fees	Informational	Fixed
I-02	apply_strategy can halt after changing strategy Coin	Informational	Fixed

I-03	Incoherent dynamic field addition when revoking pool manager	Informational	Fixed
I-04	Missing natspec for the pair parameter in position manager module	Informational	Fixed
I-05	Unnecessary if/else statement during fee withdrawal	Informational	Fixed

Medium Severity Issues

M-01: An attacker can steal un-swapped strategy fees using swap router's refund mechanism

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: tradeport_gold_strategy/sources/tradeport_gold_strategy.move#L157	Status: Fixed	

Description:

The issue assumes USDC fees accumulate in the system.

The `apply_strategy` function uses the `swap_router` to swap `USDC` strategy fees for `XAUM`.

The router's underlying swap helpers (`swap_exact_x_to_y`, `swap_exact_y_to_x`) settle the swap by paying only the amount actually consumed, then unconditionally call `utils::refund(coin_in, tx_context::sender(ctx))` to return any unspent input tokens to the **transaction sender**.

Since `apply_strategy` is permissionless, an attacker can manipulate the pool state to cause a partial swap. For example, by draining liquidity and making the pool reach a state where no active liquidity remains in that direction.

This can also happen naturally in a pool with low liquidity.

The swap consumes only a fraction of the provided `USDC`. The router then refunds the remaining strategy `USDC` directly to the attacker (the transaction sender), not back to the `Manager` object.

Recommendations:

Implement an additional function in the router that would return the unspent `Balance` as well so it can be handled explicitly (e.g. sent to the protocol treasury).

Customer Response:

Fixed in commit [ed615aa](#).

Fix Review:

Fix confirmed.

M-02: Protocol fees are wrongly determined resulting in both liquidity providers and protocol being credited the wrong portion of the swap fees

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: tradeport_strategy_clmm/sources/pool.move#L563	Status: Fixed	

Description:

The bug stems in the `pool::swap`, specifically when determining the protocol's cut applied on the swap fees collected on the input Coin.

The fees are applied on the input Coin and the protocol's fee rate should also be determined based on the input Coin.

However, the relevant `if` statement used to determine which rate (`x`'s rate or `y`'s rate) to apply uses `exact_in` rather than `x_for_y`.

This means if a user attempts to swap `CoinY` for `CoinX` using `exact_in == true` (specifying the amount of `CoinY` to input), the logic will apply `x`'s protocol fee rate.

As a result, both the liquidity providers and the protocol will be credited an incorrect amount of swap fee.

Recommendations:

To determine the right protocol fee rate, use `x_for_y` rather than `exact_in`.

Customer Response:

Fixed in commit [ed615aa](#).

Fix Review:

Fix confirmed.

M-03: Strategy balance swap can be sandwiched due to the function being permissionless

Severity: Medium	Impact: Medium	Likelihood: Low
Files: tradeport_gold_strategy/sources/tradeport_gold_strategy.move#L157	Status: Fixed	

Description:

The `apply_strategy` function is permissionless and pulls strategy fees from the configured `GLDSTR/USDC` pool before swapping all these `USDC` for `XAUM` using the CLMM router.

The issue is that the `swap_exact_input` call is missing every slippage parameters (`amount_out_min`, `sqrt_price_limit`, `deadline`) which allows the swap to succeed even when the exchange rate is economically unfavorable and exploitable.

As a result, an attacker is able to perform a swap from `USDC` to `XAUM`, to move the price of `XAUM` upward, execute `apply_strategy` to push the `XAUM` price even higher (potentially converting the input `USDC` to dust of `XAUM`) before executing a swap in the opposite direction (selling `XAUM` at the increased exchange rate) to make a profit.

This is known as a sandwich attack.

Recommendations:

Make the function admin restricted with slippage parameters (`amount_out_min`, `sqrt_price_limit`, `deadline`) to prevent the swap from succeeding in case the minimum amount of `XAUM` out is not met or the price limit is breached.

Customer Response:

Until `apply_strategy` is public we can't fix it completely. We will be running a backend service which will keep strategy fees small. Additionally we added a limiter on the amount of `USDC` that can be swapped as well as a cooldown period between each `apply_strategy` call in commit [9628288](#).



Fix Review:

Fix confirmed.

Low Severity Issues

L-01: An attacker can grief reward accrual by poking at a position regularly resulting in no rewards emitted

Severity: Low	Impact: Medium	Likelihood: Low
Files: tradeport_strategy_clmm/sources/pool.move#L1736-L1738	Status: Fixed	

Description:

The `update_reward_infos` function in the `pool.move` module is called every time a position is updated before being modified.

In case the `time_delta` and `reward_per_seconds` are small enough, their product (`pending_reward`) could be lower than `self.liquidity` and produce a division resulting in 0.

```
fun update_reward_infos<X, Y>(self: &mut Pool<X, Y>, current_timestamp: u64):  
vector<u128> {  
  // --- snip ---  
  if (current_timestamp > reward_info.last_update_time) {  
    let latest_update_timestamp = math::min(current_timestamp,  
reward_info.ended_at_seconds);  
    if (self.liquidity != 0 && latest_update_timestamp >  
reward_info.last_update_time) {  
      let time_delta = latest_update_timestamp -  
reward_info.last_update_time;  
@>      let pending_reward = full_math_u128::full_mul((time_delta as u128),  
reward_info.reward_per_seconds);  
@>      let reward_growth_delta = pending_reward / (self.liquidity as u256);  
      reward_info.reward_growth_global = full_math_u128::wrapping_add(  
        reward_info.reward_growth_global,  
        (reward_growth_delta as u128)
```

```
    );  
    reward_info.total_reward_allocated =  
reward_info.total_reward_allocated + ((pending_reward / (constants::get_q64() as  
u256)) as u64);  
    };  
    reward_info.last_update_time = current_timestamp;  
};  
vector::push_back(&mut reward_growths_global,  
reward_info.reward_growth_global);  
// --- snip ---
```

This results in no rewards accrued, but (the important part) `last_update_time` still gets updated, meaning the accumulation for that period is lost.

Anyone with a `Position` can call `modify_liquidity` with `O liquidity_delta` just to update it rapidly at small intervals so that `time_delta` would always be the smallest possible value and hence prevent reward accrual.

Recommendations:

This is easily solved by ensuring `reward_growth_delta` is greater than 0 and only then update `last_update_time`.

Customer Response:

Fixed in commit [ed615aa](#).

Fix Review:

Fix confirmed.

L-02: Strategy fee internal swap will be affected by the initial swap's price impact

Severity: Low	Impact: Low	Likelihood: Low
Files: tradeport_strategy_clmm/sources/pool.move#L907 tradeport_strategy_clmm/sources/pool.move#L940	Status: Acknowledged	

Description:

The issue happens when the strategy mode is configured such that the **strategy fee** is charged on one side while the **strategy coin** is set to the other side.

It occurs in the following configuration patterns:

- STRATEGY_MODE_X_ONLY + strategy_coin_x = false on X -> Y swaps.
- STRATEGY_MODE_Y_ONLY + strategy_coin_x = true on Y -> X swaps.
- STRATEGY_MODE_X_AND_Y + strategy_coin_x = true on Y -> X swaps.
- STRATEGY_MODE_X_AND_Y + strategy_coin_x = false on X -> Y swaps.

When a swap pays a strategy fee in **Coin X**, but the strategy is configured to hold **Coin Y**, the pool immediately converts that fee from **X** to **Y** inside the same flow.

That means one user swap can effectively cause two swaps in the same direction:

1. the user's original swap
2. the fee-conversion swap

Because the second swap happens after the first has already moved the price, it usually executes at a "worse" price. This can increase cost in low-liquidity or narrow-range conditions and potentially result in less strategy fees being accrued.

Recommendations:

In order to maximize the strategy fee conversion, the protocol can implement an admin restricted function responsible for performing the strategy fee swap when the pool conditions are more profitable.

Customer Response:

Acknowledged as this is a design decision.

L-03: Router exact output functions fail to ensure the output coin value matches the expected value

Severity: Low	Impact: Low	Likelihood: Low
Files: tradeport_strategy_clmm/sources/swap_router.move#L142-L144 tradeport_strategy_clmm/sources/swap_router.move#L178-L180	Status: Fixed	

Description:

The `swap_x_to_exact_y` and `swap_y_to_exact_x` functions in the `swap_router` allow passing a `sqrt_price_limit` parameter, denoting a price limit which, if reached during the swap, halts the execution and returns the value of output `Balance` calculated.

However, passing a `sqrt_price_limit` equal to 0 (or the highest/lowest tick) means there is no price limit. Doing this, the user expects to receive exactly the `Balance` value (`amount_x_out` or `amount_y_out`) provided as parameter, regardless of where the price ends up.

This is a behavior that is implemented on Uniswap V3 which the FlowX CLMM is based on and is thus missing.

Recommendations:

In order for the function to be consistent with its name and compliant with Uniswap V3, both `swap_x_to_exact_y` and `swap_y_to_exact_x` functions must verify the output `Balance` value against the supplied `amount_x/y_out` when `sqrt_price_limit == 0`.

```
public fun swap_x_to_exact_y<X, Y>(
  // --- snip ---
  if (amount_in_required > coin::value(&coin_in)) {
    abort E_EXCESSIVE_INPUT_AMOUNT
  };
+ if(sqrt_price_limit == 0 && balance::value(&y_out) != amount_y_out) {
```

```
+   abort E_INSUFFICIENT_OUTPUT_AMOUNT
+ };

public fun swap_y_to_exact_x<X, Y>(
  // --- snip ---
  if (amount_in_required > coin::value(&coin_in)) {
    abort E_EXCESSIVE_INPUT_AMOUNT
  };
+ if(sqrt_price_limit == 0 && balance::value(&x_out) != amount_x_out) {
+   abort E_INSUFFICIENT_OUTPUT_AMOUNT
+ };
```

Customer Response:

Fixed in commit [ed615aa](#).

Fix Review:

Fix confirmed.

Informational Severity Issues

I-01: Un-swapped strategy payment will be put in reserves when paying strategy fees

Files:

tradeport_strategy_clmm/sources/pool.move#L891

Description:

In `pay_strategy_fees`, if the internal fee conversion swap only partially fills (due to hitting the price limit or due to lack of liquidity), the full input balance (`strategy_payment`) is passed to `pay`.

The `pay` function joins the entire balance into pool reserves, and since the swap receipt's debt only covers the consumed portion, the excess is donated to pool reserves (as surplus, not for LPs).

Recommendations:

Implement a mechanism to detect the un-swapped amount in and handle it explicitly, for example by sending it to the protocol treasury.

Customer Response:

Fixed in commit [ed615aa](#).

Fix Review:

Fix confirmed.

I-02: apply_strategy can halt after changing strategy Coin

Files:

tradeport_gold_strategy/sources/tradeport_gold_strategy.move#L148

Description:

The `apply_strategy` function ensures the strategy balance withdrawn from the `GLDSTR/USDC` pool only pulls `USDC` and the value of the `GLDSTR` coin equals 0 before attempting to destroy it.

However, in case the `strategy_coin_x` was misconfigured then reconfigured correctly after, a malicious user could have performed a swap to generate `GLDSTR` strategy fees that would be sitting in the pool's `strategy_reserve_x/y`. Then, when calling `apply_strategy`, the module would withdraw a `GLDSTR` coin with a value greater than 0, triggering the assertion and making the transaction abort.

Scenario:

1. `strategy_coin_x` -> `GLDSTR` (misconfiguration)
2. malicious user performs a swap that accrues strategy fees
3. fees accumulate (or are converted) into `GLDSTR` strategy reserve
4. admin notices the misconfiguration and switches `strategy_coin_x` -> `USDC`
5. future PTBs with `pool::swap + tradeport_gold_strategy::apply_strategy` will withdraw both non-zero `USDC` and `GLDSTR` and abort

Recommendations:

Handle the `GLDSTR` coin withdrawn in case its value is greater than 0.

Customer Response:

Fixed in commit [ed615aa](#).

Fix Review:

Fix confirmed.

I-03: Incoherent dynamic field addition when revoking pool manager

Files:

tradeport_strategy_clmm/sources/pool_manager.move#L151-L153

Description:

Revoking a pool manager removes an `address` stored in a `VecSet` attached to a `DynamicField` on the `PoolRegistry`, denoted by the `PoolManagerDfKey` key.

In case the `DynamicField` doesn't exist, the function creates it, with an empty `VecSet` then immediately attempts to remove an entry in it which will abort.

This check complicates the understanding of the function with unnecessary logic.

Recommendations:

Remove the statement responsible for adding the `DynamicField` in the `revoke_pool_manager` function.

Customer Response:

Fixed in commit [ed615aa](#) and enhanced in commit [5945af6](#).

Fix Review:

Fix confirmed.

I-04: Missing natspec for the pair parameter in position manager module

Files:

tradeport_strategy_clmm/sources/position_manager.move#L92

Description:

The project introduces a `pair` field in the `position::Position` structure which `position_manager::open_position` sets. However this function has a natspec for all the parameters except for `pair`.

Recommendations:

Add the natspec for the `pair` parameter.

Customer Response:

Fixed in commit [ed615aa](#).

Fix Review:

Fix confirmed, the parameter has been removed.

I-05: Unnecessary if/else statement during fee withdrawal

Files:

tradeport_gold_strategy/sources/tradeport_gold_strategy.move#L243

Description:

Both the `pay::keep` and `transfer::public_transfer` functions will transfer the fees to the `recipient` provided as parameter. This means the `if/else` statement adds unnecessary complexity to the code.

Recommendations:

Remove the `if/else` statement and transfer the fees directly to the `recipient` using `transfer::public_transfer`.

Customer Response:

Fixed in commit [ed615aa](#).

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.